

Dataset Download Directly From Json (Simulate API)

```
#Prepare Python Environment
!pip install kaggle --trusted-host pypi.org --trusted-host files.pythonhosted.org
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: jupyter in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.19.3)
Requirement already satisfied: kagglesdk<1.0,>=0.1.20 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.23)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.2)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (4.2.0)
Requirement already satisfied: mdit-py-plugins in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (0.6.1)
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (5.10.4)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (6.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.15)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.5.20)
Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=1.0->jupyter->kaggle)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle) (4.2)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle)
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle) (5.7)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core!=5.0.*,>=4.12->nbformat)
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from referencing>=0.28.4->jupyter)
```

```
#Execute Command For Download Datasets.
import os
os.environ['KAGGLE_USERNAME'] = 'renatosilvaportilla' #Use Own User ID
os.environ['KAGGLE_KEY'] = '51d911d914902495e893339ee7acbf04' #Use Own Kaggle Token

!kaggle datasets download -d olistbr/brazilian-ecommerce --unzip -p data/raw/
```

```
Dataset URL: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
License(s): CC-BY-NC-SA-4.0
Downloading brazilian-ecommerce.zip to data/raw
100% 42.6M/42.6M [00:00<00:00, 252MB/s]
```

Extract Data

```
# Load each CSV file into a pandas DataFrame
# os.path.join handles the folder + filename combination safely across operating systems
# Each variable represents one table in our future database

import pandas as pd
raw_path = r"C:\Users\YOGA\Downloads\Brazilian_Ecommerce_Olys.ipynb"

orders = pd.read_csv(os.path.join(raw_path, "olist_orders_dataset.csv"))
customers = pd.read_csv(os.path.join(raw_path, "olist_customers_dataset.csv"))
order_items = pd.read_csv(os.path.join(raw_path, "olist_order_items_dataset.csv"))
products = pd.read_csv(os.path.join(raw_path, "olist_products_dataset.csv"))
sellers = pd.read_csv(os.path.join(raw_path, "olist_sellers_dataset.csv"))
payments = pd.read_csv(os.path.join(raw_path, "olist_order_payments_dataset.csv"))
reviews = pd.read_csv(os.path.join(raw_path, "olist_order_reviews_dataset.csv"))
geolocation = pd.read_csv(os.path.join(raw_path, "olist_geolocation_dataset.csv"))
category_translation = pd.read_csv(os.path.join(raw_path, "product_category_name_translation.csv"))

print("All 9 files loaded!")
```

```

# Store all DataFrames in a dictionary so we can loop through them cleanly
# This avoids repeating the same code 9 times for each table
datasets = {
    "orders": orders,
    "customers": customers,
    "order_items": order_items,
    "products": products,
    "sellers": sellers,
    "payments": payments,
    "reviews": reviews,
    "geolocation": geolocation,
    "category_translation": category_translation
}

# Loop through each table and print a summary
for name, df in datasets.items():
    print(f"\n--- {name} ---")

    # shape[0] = number of rows, shape[1] = number of columns
    print(f"Rows: {df.shape[0]} | Columns: {df.shape[1]}")

    # isnull().sum() counts empty cells per column
    # We filter to only show columns that actually have nulls
    nulls = df.isnull().sum()
    nulls = nulls[nulls > 0]

    if len(nulls) > 0:
        print(f"Nulls:\n{nulls}")
    else:
        print("No nulls found")

```

```

--- orders ---
Rows: 99441 | Columns: 8
Nulls:
order_approved_at          160
order_delivered_carrier_date 1783
order_delivered_customer_date 2965
dtype: int64

--- customers ---
Rows: 99441 | Columns: 5
No nulls found

--- order_items ---
Rows: 112650 | Columns: 7
No nulls found

--- products ---
Rows: 32951 | Columns: 9
Nulls:
product_category_name      610
product_name_lenght        610
product_description_lenght  610
product_photos_qty         610
product_weight_g           2
product_length_cm          2
product_height_cm          2
product_width_cm           2
dtype: int64

--- sellers ---
Rows: 3095 | Columns: 4
No nulls found

--- payments ---
Rows: 103886 | Columns: 5
No nulls found

--- reviews ---
Rows: 99224 | Columns: 7
Nulls:
review_comment_title      87656
review_comment_message    58247
dtype: int64

--- geolocation ---
Rows: 1000163 | Columns: 5
No nulls found

--- category_translation ---

```

Rows: 71 | Columns: 2
No nulls found

▼ Transform Datasets

• Null Values

```
# --- ORDERS ---
# These nulls are valid – order not yet approved/shipped/delivered
# No action needed, we leave them as is
print("Orders: nulls left intentionally because they represent pending order statuses")

# --- PRODUCTS ---
# Fill missing category and text metadata with neutral defaults
products["product_category_name"] = products["product_category_name"].fillna("unknown")
products["product_name_lenght"] = products["product_name_lenght"].fillna(0)
products["product_description_lenght"] = products["product_description_lenght"].fillna(0)
products["product_photos_qty"] = products["product_photos_qty"].fillna(0)

# Fill missing physical dimensions with median – only 2 rows affected
# Median is better than average here because it's not skewed by outliers
products["product_weight_g"] = products["product_weight_g"].fillna(products["product_weight_g"].median())
products["product_length_cm"] = products["product_length_cm"].fillna(products["product_length_cm"].median())
products["product_height_cm"] = products["product_height_cm"].fillna(products["product_height_cm"].median())
products["product_width_cm"] = products["product_width_cm"].fillna(products["product_width_cm"].median())

print("Products: nulls filled successfully")

# --- REVIEWS ---
# Most customers skip writing a title or message – fill with neutral text
reviews["review_comment_title"] = reviews["review_comment_title"].fillna("No title")
reviews["review_comment_message"] = reviews["review_comment_message"].fillna("No comment")

print("Reviews: nulls filled successfully")

# --- VERIFY ---
# Confirm no unwanted nulls remain
print("\n— Null check after cleaning —")
for name, df in datasets.items():
    total_nulls = df.isnull().sum().sum()
    if total_nulls > 0:
        print(f"{name}: {total_nulls} nulls remaining")
    else:
        print(f"{name}: clean")
```

Orders: nulls left intentionally because they represent pending order statuses
Products: nulls filled successfully
Reviews: nulls filled successfully

— Null check after cleaning —
orders: 4908 nulls remaining
customers: clean
order_items: clean
products: clean
sellers: clean
payments: clean
reviews: clean
geolocation: clean
category_translation: clean

• Split 75/25 df for dev and test

```
# Split orders into 75% development set and 25% test set
# random_state=42 ensures the split is always identical every run – reproducible

orders_dev = orders.sample(frac=0.75, random_state=42)
orders_test = orders.sample(frac=0.25, random_state=42)

print(f"Total orders: {len(orders)}")
print(f"Dev set: {len(orders_dev)} orders (75%)")
print(f"Test set: {len(orders_test)} orders (25%)")
```

Total orders: 99441
Dev set: 74581 orders (75%)
Test set: 24860 orders (25%)

```
# — FILTER OTHER TABLES TO MATCH DEV ORDERS —————
# Extract the order IDs from the dev set – this is our anchor
dev_order_ids = orders_dev["order_id"]
test_order_ids = orders_test["order_id"]

# Filter each table that connects to orders by order_id
# Tables with no order_id (products, sellers, geolocation, category) stay fully loaded

# DEV filters
customers_dev = customers[customers["customer_id"].isin(orders_dev["customer_id"])]
order_items_dev = order_items[order_items["order_id"].isin(dev_order_ids)]
payments_dev = payments[payments["order_id"].isin(dev_order_ids)]
reviews_dev = reviews[reviews["order_id"].isin(dev_order_ids)]

# TEST filters
customers_test = customers[customers["customer_id"].isin(orders_test["customer_id"])]
order_items_test = order_items[order_items["order_id"].isin(test_order_ids)]
payments_test = payments[payments["order_id"].isin(test_order_ids)]
reviews_test = reviews[reviews["order_id"].isin(test_order_ids)]

# Products, sellers, geolocation, category_translation have no order_id
# They are reference tables – shared across both dev and test
print("Filtering complete")
print(f"\nDEV SET:")
print(f"  Orders: {len(orders_dev)}")
print(f"  Customers: {len(customers_dev)}")
print(f"  Order Items: {len(order_items_dev)}")
print(f"  Payments: {len(payments_dev)}")
print(f"  Reviews: {len(reviews_dev)}")

print(f"\nTEST SET:")
print(f"  Orders: {len(orders_test)}")
print(f"  Customers: {len(customers_test)}")
print(f"  Order Items: {len(order_items_test)}")
print(f"  Payments: {len(payments_test)}")
print(f"  Reviews: {len(reviews_test)}")
```

Filtering complete

DEV SET:
 Orders: 74581
 Customers: 74581
 Order Items: 84439
 Payments: 77931
 Reviews: 74410

TEST SET:
 Orders: 24860
 Customers: 24860
 Order Items: 28143
 Payments: 26060
 Reviews: 24785

```
# — SAVE DEV AND TEST SETS —————
# Define folder paths for dev and test sets
dev_path = r"C:\Users\Renato\Documents\Graduate Education\UE\Second Term\Data Engeneer\E_commerce Brazilian Project\data\dev"
test_path = r"C:\Users\Renato\Documents\Graduate Education\UE\Second Term\Data Engeneer\E_commerce Brazilian Project\data\test"

# Create folders if they don't exist
os.makedirs(dev_path, exist_ok=True)
os.makedirs(test_path, exist_ok=True)

# — SAVE DEV SET —————
# Transaction tables – filtered to 75% of orders
orders_dev.to_csv(os.path.join(dev_path, "orders_dev.csv"), index=False)
customers_dev.to_csv(os.path.join(dev_path, "customers_dev.csv"), index=False)
order_items_dev.to_csv(os.path.join(dev_path, "order_items_dev.csv"), index=False)
payments_dev.to_csv(os.path.join(dev_path, "payments_dev.csv"), index=False)
reviews_dev.to_csv(os.path.join(dev_path, "reviews_dev.csv"), index=False)

# Reference tables – fully loaded in both dev and test
products.to_csv(os.path.join(dev_path, "products_dev.csv"), index=False)
sellers.to_csv(os.path.join(dev_path, "sellers_dev.csv"), index=False)
geolocation.to_csv(os.path.join(dev_path, "geolocation_dev.csv"), index=False)
category_translation.to_csv(os.path.join(dev_path, "category_translation_dev.csv"), index=False)
```

```

print("DEV set saved successfully")

# — SAVE TEST SET —————
# Transaction tables – filtered to 25% of orders
orders_test.to_csv(os.path.join(test_path, "orders_test.csv"), index=False)
customers_test.to_csv(os.path.join(test_path, "customers_test.csv"), index=False)
order_items_test.to_csv(os.path.join(test_path, "order_items_test.csv"), index=False)
payments_test.to_csv(os.path.join(test_path, "payments_test.csv"), index=False)
reviews_test.to_csv(os.path.join(test_path, "reviews_test.csv"), index=False)

# Reference tables – same full tables for test environment
products.to_csv(os.path.join(test_path, "products_test.csv"), index=False)
sellers.to_csv(os.path.join(test_path, "sellers_test.csv"), index=False)
geolocation.to_csv(os.path.join(test_path, "geolocation_test.csv"), index=False)
category_translation.to_csv(os.path.join(test_path, "category_translation_test.csv"), index=False)

print("TEST set saved successfully")

# — VERIFY FILES WERE CREATED —————
print("\n— DEV folder contents —")
for f in os.listdir(dev_path):
    print(f" {f}")

print("\n— TEST folder contents —")
for f in os.listdir(test_path):
    print(f" {f}")

```

```

DEV set saved successfully
TEST set saved successfully

```

```

— DEV folder contents —
category_translation_dev.csv
customers_dev.csv
geolocation_dev.csv
orders_dev.csv
order_items_dev.csv
payments_dev.csv
products_dev.csv
reviews_dev.csv
sellers_dev.csv

```

```

— TEST folder contents —
category_translation_test.csv
customers_test.csv
geolocation_test.csv
orders_test.csv
order_items_test.csv
payments_test.csv
products_test.csv
reviews_test.csv
sellers_test.csv

```

▼ Load Datasets

```

#On normal case scenario default port is 5432 for my specific case the port is 5432
from sqlalchemy import create_engine

# Try port 5432 first (default)
try:
    engine = create_engine('postgresql://postgres:12345@localhost:5432/postgres') #Instead of '12345' paste your own password.
    with engine.connect() as connection:
        print("Connected on port 5432")
except:
    print("Port 5432 failed, trying 5433...")
    engine = create_engine('postgresql://postgres:12345@localhost:5433/postgres') #Instead of '12345' paste your own password.
    with engine.connect() as connection:
        print("Connected on port 5433")

```

```

Connected on port 5432

```

```

from sqlalchemy import create_engine

# — CONNECT TO OLIST DATABASE —————
# Port 5432 = PostgreSQL 18 on your machine
# olist_ecommerce = your project database

engine = create_engine('postgresql://postgres:12345@localhost:5432/olist_ecommerce')

```

```
with engine.connect() as connection:
    print("Connected to olist_ecommerce successfully!")
```

Connected to olist_ecommerce successfully!

```
from sqlalchemy import text

# — CREATE SCHEMA —————
# We define all 9 tables with their columns, data types, and keys
# ORDER MATTERS – reference tables must be created before transaction tables
# because transaction tables have foreign keys pointing to them

schema_sql = """

-- REFERENCE TABLES FIRST (no foreign key dependencies)

-- Sellers: merchants who list products on the platform
CREATE TABLE IF NOT EXISTS sellers (
    seller_id VARCHAR PRIMARY KEY,
    seller_zip_code_prefix VARCHAR,
    seller_city VARCHAR,
    seller_state VARCHAR
);

-- Customers: people who placed orders
CREATE TABLE IF NOT EXISTS customers (
    customer_id VARCHAR PRIMARY KEY,
    customer_unique_id VARCHAR,
    customer_zip_code_prefix VARCHAR,
    customer_city VARCHAR,
    customer_state VARCHAR
);

-- Category Translation: Portuguese to English category names
CREATE TABLE IF NOT EXISTS category_translation (
    product_category_name VARCHAR PRIMARY KEY,
    product_category_name_english VARCHAR
);

-- Products: items listed for sale on the platform
CREATE TABLE IF NOT EXISTS products (
    product_id VARCHAR PRIMARY KEY,
    product_category_name VARCHAR,
    product_name_lenght FLOAT,
    product_description_lenght FLOAT,
    product_photos_qty FLOAT,
    product_weight_g FLOAT,
    product_length_cm FLOAT,
    product_height_cm FLOAT,
    product_width_cm FLOAT,
    FOREIGN KEY (product_category_name) REFERENCES category_translation(product_category_name)
);

-- Geolocation: zip code mappings to coordinates and city
CREATE TABLE IF NOT EXISTS geolocation (
    geolocation_zip_code_prefix VARCHAR,
    geolocation_lat FLOAT,
    geolocation_lng FLOAT,
    geolocation_city VARCHAR,
    geolocation_state VARCHAR
);

-- TRANSACTION TABLES NEXT (depend on reference tables)

-- Orders: every purchase made on the platform
CREATE TABLE IF NOT EXISTS orders (
    order_id VARCHAR PRIMARY KEY,
    customer_id VARCHAR,
    order_status VARCHAR,
    order_purchase_timestamp TIMESTAMP,
    order_approved_at TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
    order_delivered_customer_date TIMESTAMP,
    order_estimated_delivery_date TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

```

-- Order Items: products inside each order
CREATE TABLE IF NOT EXISTS order_items (
    order_id VARCHAR,
    order_item_id INTEGER,
    product_id VARCHAR,
    seller_id VARCHAR,
    shipping_limit_date TIMESTAMP,
    price FLOAT,
    freight_value FLOAT,
    PRIMARY KEY (order_id, order_item_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (seller_id) REFERENCES sellers(seller_id)
);

-- Payments: how each order was paid
CREATE TABLE IF NOT EXISTS payments (
    order_id VARCHAR,
    payment_sequential INTEGER,
    payment_type VARCHAR,
    payment_installments INTEGER,
    payment_value FLOAT,
    PRIMARY KEY (order_id, payment_sequential),
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

-- Reviews: customer feedback after delivery
CREATE TABLE IF NOT EXISTS reviews (
    review_id VARCHAR PRIMARY KEY,
    order_id VARCHAR,
    review_score INTEGER,
    review_comment_title VARCHAR,
    review_comment_message TEXT,
    review_creation_date TIMESTAMP,
    review_answer_timestamp TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
"""

# Execute the schema
with engine.connect() as connection:
    connection.execute(text(schema_sql))
    connection.commit()
    print("All 9 tables created successfully!")

```